Faculté des sciences

Département d'informatique

TD 1

Structure des données

IA S3
2026/2027

Exercice 1 - Structure `Etudiant`

Un étudiant est caractérisé par un numéro, un nom de 20 caractères au maximum et une moyenne.

1. Définir la structure struct Etudiant.
2. Déclarer et initialiser un étudiant.
3. Écrire les fonctions :

```
struct Etudiant saisirEtudiant(void);
void afficherEtudiant(struct Etudiant e);
int estAdmis(struct Etudiant e);
```

Un étudiant est admis si sa moyenne est supérieure ou égale à 10.

4. Vérifier que la moyenne saisie appartient à [0, 20].
5. Écrire un programme principal utilisant ces fonctions.

Exercice 2 - Tableau de structures

On souhaite gérer au maximum 100 étudiants à l'aide d'un tableau de structures.

1. Saisir n étudiants, avec $1 \leq n \leq 100$.
2. Afficher les étudiants admis.
3. Calculer la moyenne de la classe.
4. Afficher tous les étudiants ayant la meilleure moyenne.
5. Rechercher un étudiant par son numéro.
6. Donner la complexité de chaque traitement.

Exercice 3 - Structures et pointeurs

On considère :

```
typedef struct {
    int numero;
    char nom[40];
    int quantite;
    float prix;
} Article;
```

1. Déclarer un article a et un pointeur p initialisé avec son adresse.
2. Affecter une quantité et un prix à l'article en utilisant p.
3. Réécrire les accès avec la notation (*p).champ.
4. Écrire :

```
int modifierPrix(Article *a, float nouveauPrix);
int ajouterStock(Article *a, int quantite);
int vendre(Article *a, int quantite);
```

Les fonctions retournent 1 si l'opération réussit, sinon 0. Une vente ne doit pas rendre le stock négatif.

5. Expliquer pourquoi une fonction recevant seulement une copie de l'article ne modifie pas directement l'article du programme appelant.

Exercice 4 - Structures imbriquées

Un congé comporte un nom d'employé, une date de début et une date de fin.

1. Définir les types Date et Conge avec typedef.
2. Écrire une fonction :

```
int comparerDates(Date a, Date b);
```

Elle retourne -1, 0 ou 1, selon que a précède, égale ou suit b.

3. Écrire une fonction vérifiant que la fin d'un congé ne précède pas son début.
4. Déclarer un tableau de 20 congés.
5. Afficher les congés commençant pendant un mois et une année donnés.

Pour cet exercice, on suppose que les dates fournies sont valides.

Exercice 5 - typedef et structure récursive

A. Synonymes de types

On donne :

```
typedef int Entier;
typedef int *PtrEntier;
typedef float TableauNotes[5];
```

1. Déclarer deux entiers, deux pointeurs et un tableau de notes avec ces synonymes.
2. Comparer :

```
int *p1, p2;    et   PtrEntier p1, p2;
```

3. Calculer la moyenne d'un objet de type TableauNotes.

B. Structure récursive

Définir une structure Individu contenant un nom, un âge et deux pointeurs vers ses parents.

4. Créer trois individus : un enfant, son père et sa mère.
5. Relier l'enfant à ses parents.
6. Afficher leurs noms à partir de l'enfant.
7. Expliquer pourquoi les champs représentant les parents doivent être des pointeurs et non des objets Individu directement contenus dans la structure.

Exercice 6 - Unions et énumérations

Une valeur peut être un entier, un réel ou un caractère.

1. Définir une énumération TypeValeur contenant ENTIER, REEL et CARACTERE.
2. Définir une union Donnee contenant les trois champs correspondants.
3. Définir une structure Valeur regroupant un champ type et un champ donnee.
4. Écrire :

```
void afficherValeur(Valeur v);
```

Utiliser switch pour afficher uniquement le champ indiqué par type.

5. Créer et afficher un tableau contenant des valeurs de catégories différentes.
6. Expliquer le rôle du champ type.
7. Comparer, avec sizeof, la taille de l'union et celle d'une structure contenant simultanément les trois champs.
8. Déterminer les valeurs des constantes suivantes :

```
enum Etat {
    INITIAL,
    EN_COURS = 4,
    TERMINE,
    ERREUR = 10,
    ANNULE
};
```

Exercice 7 - Lire une spécification abstraite

On considère le TAD Vecteur du cours :

vect : Entier → Vecteur
taille : Vecteur → Entier
ième : Vecteur × Entier → Élément
changer_ième : Vecteur × Entier × Élément → Vecteur

1. Identifier les constructeurs, observateurs et transformateurs.
2. Donner les préconditions de chaque opération.
3. Expliquer pourquoi ième est une opération partielle.
4. Soit un vecteur v de taille 4 contenant : {2, 5, 8, 1}

Déterminer :

taille(v)
ième(v, 2)
ième(changer_ième(v, 1, 9), 1)
ième(changer_ième(v, 1, 9), 3)

5. Indiquer si les expressions suivantes sont définies :

vect(-2)
ième(v, 4)
changer_ième(v, 0, 7)

6. Formuler l'axiome exprimant que la modification d'un élément ne change pas la taille.

Exercice 8 - Concevoir un TAD Compteur

Un compteur contient un entier positif ou nul. Il peut être créé, consulté, incrémenté et décrémenté.

1. Rédiger sa spécification avec les rubriques :
   Type, Utilise, Opérations, Préconditions et Axiomes.
2. Utiliser les opérations :

creer : → Compteur
valeur : Compteur → Entier
incrementer : Compteur → Compteur
decrementer : Compteur → Compteur

3. Classer les opérations par catégorie.
4. Donner la précondition de decrementer.
5. Écrire les axiomes décrivant la valeur initiale et les effets des transformations.
6. Les deux axiomes suivants peuvent-ils appartenir à une même spécification cohérente ? Justifier.

valeur(creer()) = 0
valeur(creer()) = 1

Exercice 9 - Module booléen et compilation séparée

Réaliser le TAD booléen à l'aide de trois fichiers : booleen.h, booleen.c et main.c.

1. Dans le fichier d'en-tête, définir :

typedef enum { faux, vrai } Booleen;

et déclarer les opérations et, ou et non.

2. Protéger le fichier d'en-tête contre les inclusions multiples.
3. Implémenter les trois opérations dans booleen.c.
4. Afficher leurs tables de vérité dans main.c.
5. Vérifier pour toutes les combinaisons possibles :

non(non(a)) = a
et(vrai, a) = a
ou(a, b) = non(et(non(a), non(b)))

6. Donner les commandes permettant de compiler séparément les deux fichiers .c, puis de produire l'exécutable.
7. Distinguer prétraitement, compilation et édition de liens.
8. Expliquer l'erreur obtenue si l'on oublie le fichier objet booleen.o lors de l'édition de liens.